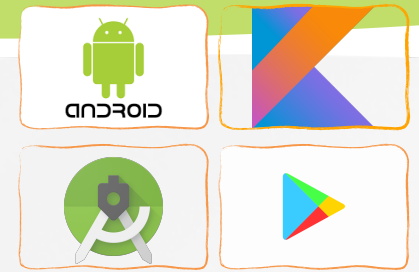




Software Development for Mobile Devices

Nguyen Le Hoang Dung
[*nlhdung@fit.hcmus.edu.vn*](mailto:nlhdung@fit.hcmus.edu.vn)

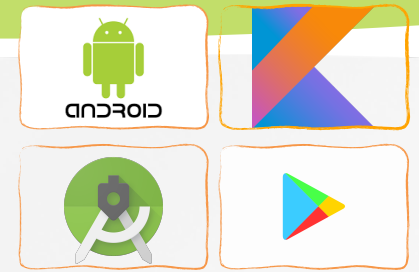


Using WebViews

Design Strategies

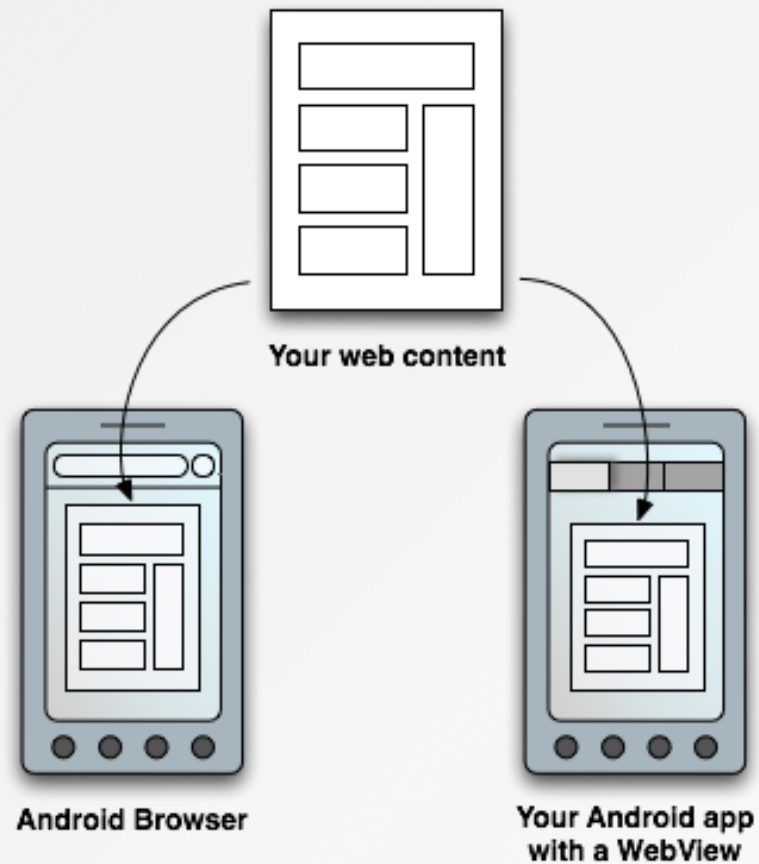
» **Pure Android:**

- A native Android app using the SDK and primitive resources such as widgets, services, activities, fragments, content-providers, notifications, etc.)
- It can uses Android, and we keep it in the device.

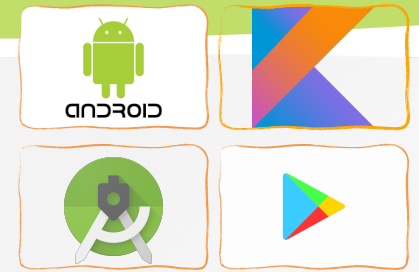


Using WebViews

Design Strategies



<https://developer.android.com/guide/webapps>



Using WebViews

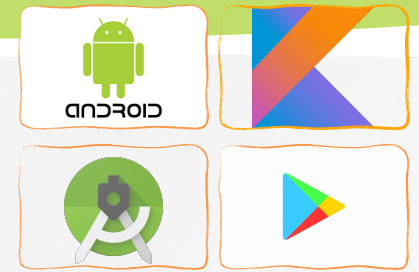
Design Strategies

» **Pure External HTML:**

- Create a remote website on any server.
- Use Android devices to fetch that external web-pages by browser.

» **Mixed Mode**

- Create an internal website hosted in the device.
- Building app which accesses that local HTML pages with local Android resources



Using WebViews

Pros and Cons

» **Pure Android:**

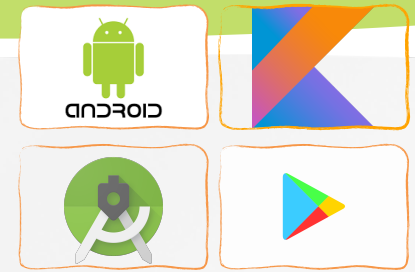
- Rich UI and logic controls but it's limited to Android.
- Porting to iOS by re-writing of all its parts!

» **Pure External HTML:**

- The best option for porting application
- Interact with the external website with Internet in any devices
- It's limited to use any of the multiple Android's hardware and software resources

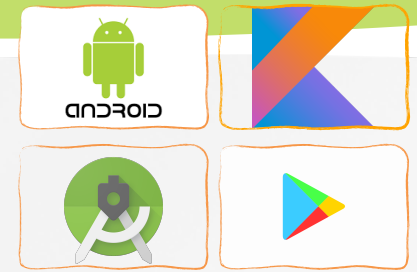
» **Mixed Mode**

- An interesting in-between compromise
- Some screens with Rich UI
- At least in principle, this is more 'easily' transferred from one platform to the other

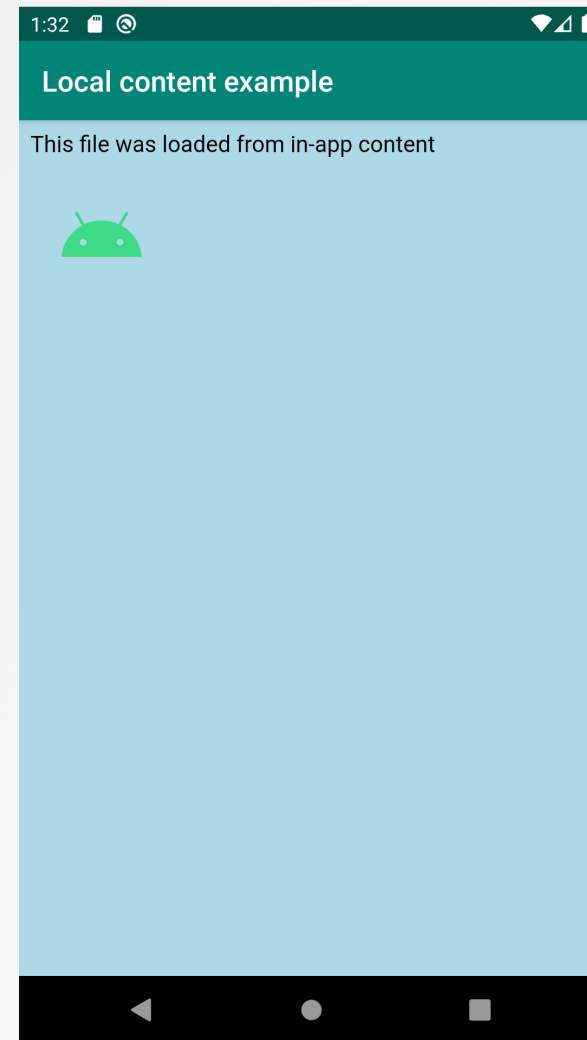
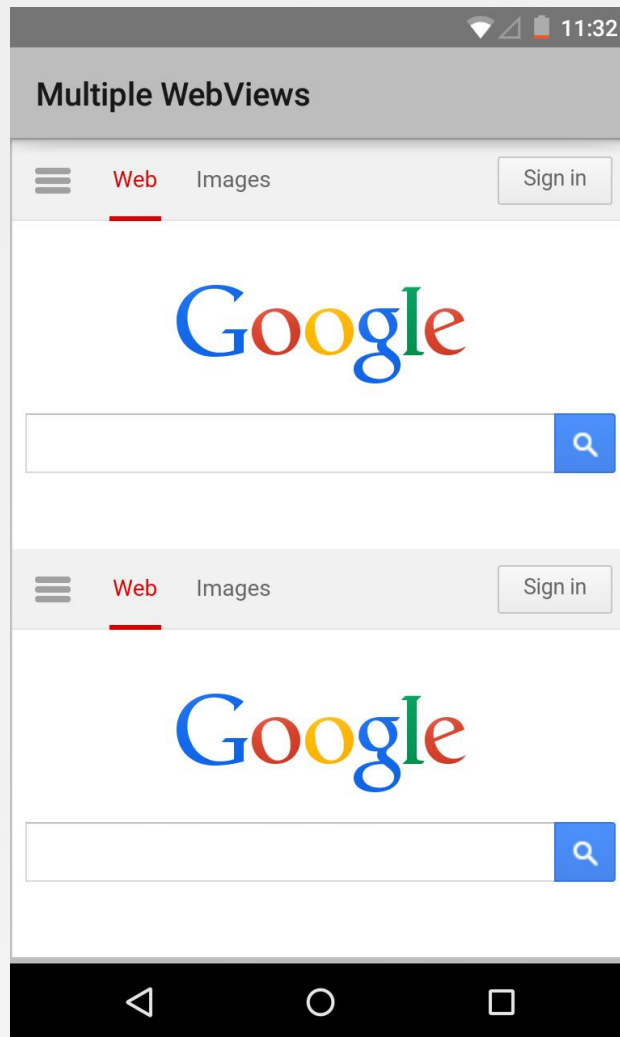


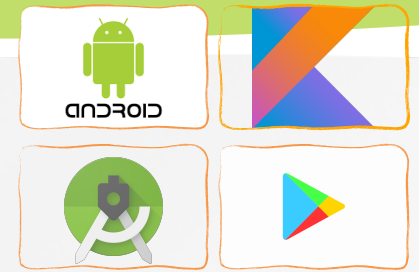
WebView Features

- » WebViews use the WebKit Open Source, a [web browser engine](#) (it's also used by macOS, iOS on Safari, Mail, App Store)
- » Webview is an extension of Android's view class that allows you to display web pages.
- » It does *not* include any features of a fully developed web browser, such as navigation controls or an address bar
- » Main methods:
 - Load/Reload a page, navigate forward/backward through a history record,
 - Zoom in and out,
 - Use **CSS & JavaScript features** to provide different styles and images based on the screen's pixel density.
 - *Exchange data with the Android device using a **JavaScript Interface***
 - Perform text searches, capture pictures, load data / stop loading ...
 - Implementing those methods and services could be done in various ways, for instance through Menus, ActionBar, Buttons, etc



WebView Features





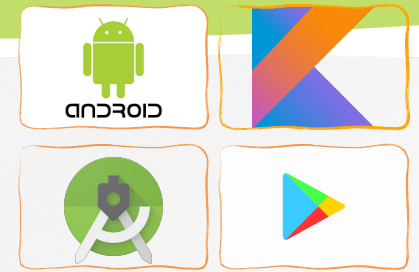
WebView Permissions

» Add the **INTERNET** permission to your Android Manifest file

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.nlhdung.webviewapp">

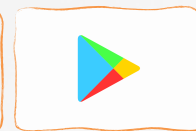
    <uses-permission android:name="android.permission.INTERNET"/>

    <application...>
</manifest>
```



WebView Examples

- » We will consider 3 different ways in which a WebView could be used:
 - A WebView is loaded with an external web-page whose URL is provided.
 - A WebView is loaded with local web-pages whose HTML page are provided.
 - A WebView is used to display an HTML page whose code is dynamically supplied by the app.

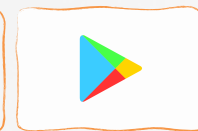


Example 1

External web-page with URL

» Main Layout

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
    <EditText
        android:id="@+id/editTextTextPersonName"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="10dp"
        android:layout_marginEnd="10dp"
        android:ems="10"
        android:hint="your url"
        android:inputType="textPersonName"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
    <Button
        android:id="@+id/backBtn"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="16dp"
        android:text="@string/web_back"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@+id/editTextTextPersonName" />
```



Example 1

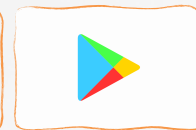
External web-page with URL

» Main Layout

```
<Button
    android:id="@+id/forwardBtn"
    android:layout_width="80dp"
    android:layout_height="wrap_content"
    android:layout_marginStart="20dp"
    android:text="@string/web_next"
    app:layout_constraintStart_toEndOf="@+id/backBtn"
    app:layout_constraintTop_toBottomOf="@+id/editTextTextPersonName" />

<Button
    android:id="@+id/reloadBtn"
    android:layout_width="48dp"
    android:layout_height="48dp"
    android:layout_marginStart="20dp"
    android:background="@android:drawable/ic_menu_rotate"
    app:layout_constraintStart_toEndOf="@+id/forwardBtn"
    app:layout_constraintTop_toBottomOf="@+id/editTextTextPersonName" />

<Button
    android:id="@+id/zoomoutBtn"
    android:layout_width="48dp"
    android:layout_height="48dp"
    android:layout_marginStart="20dp"
    android:text="-"
    app:layout_constraintStart_toEndOf="@+id/reloadBtn"
    app:layout_constraintTop_toBottomOf="@+id/editTextTextPersonName" />
```



Example 1

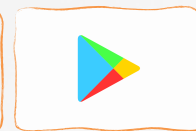
External web-page with URL

» Main Layout

```
<Button
    android:id="@+id/zoominBtn"
    android:layout_width="48dp"
    android:layout_height="48dp"
    android:layout_marginStart="20dp"
    android:layout_marginEnd="16dp"
    android:text="+"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toEndOf="@+id/zoomoutBtn"
    app:layout_constraintTop_toBottomOf="@+id/editTextTextPersonName" />
```

```
<WebView
    android:id="@+id/webView"
    android:layout_width="0dp"
    android:layout_height="0dp"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/reloadBtn" />
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
```

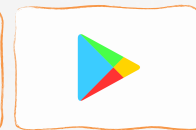


Example 1

External web-page with URL

» Main Activity

```
class MainActivity : AppCompatActivity() {  
    lateinit var webView : WebView  
    lateinit var editText: EditText  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
  
        editText = findViewById(R.id.editText)  
        webView = findViewById(R.id.webView)  
  
        // Try demo1  
        demo1TrySpecificUrl()  
    }  
  
    fun demo1TrySpecificUrl(){  
        //1  
        webView.settings.javaScriptEnabled = true  
        //2  
        webView.webViewClient = MyWebViewClient(this, editText, "hcmus.edu.vn")  
        //3  
        webView.loadUrl("https://www.hcmus.edu.vn")  
    }  
}
```



Example 1

External web-page with URL

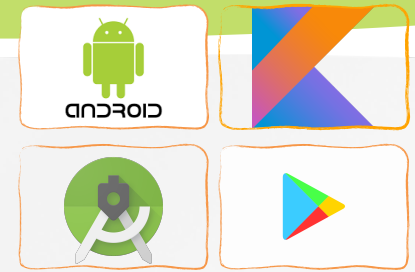
» *MyWebViewClient* class

```
private class MyWebViewClient() : WebViewClient() {
    lateinit var textView: TextView
    lateinit var hostServer: String
    lateinit var context: Context

    constructor(context: Context, textView: TextView, host: String) : this() {
        this.context = context
        this.textView = textView
        this.hostServer = host
    }

    override fun shouldOverrideUrlLoading(view: WebView, url: String): Boolean {
        Log.i("nlhdung-i", url)

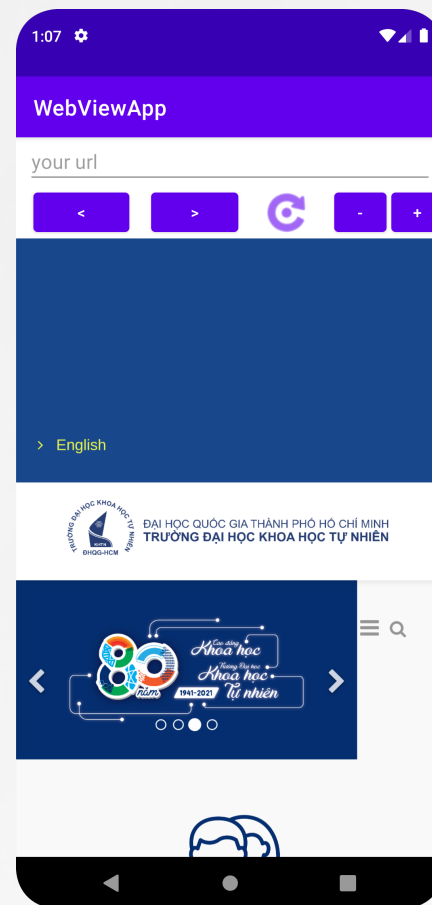
        if(url.contains(hostServer)) {
            return false
        }else{
            val intent = Intent(Intent.ACTION_VIEW, Uri.parse(url))
            context.startActivity(intent)
            return true
        }
    }
}
```

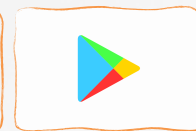


Example 1

External web-page with URL

» *Run app without handling buttons*



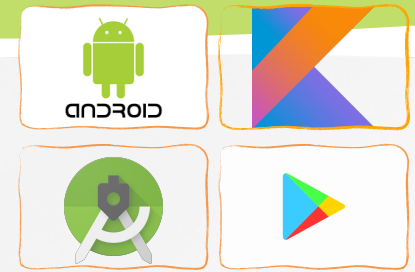


Example 1

External web-page with URL

» Handling buttons in Main Activity

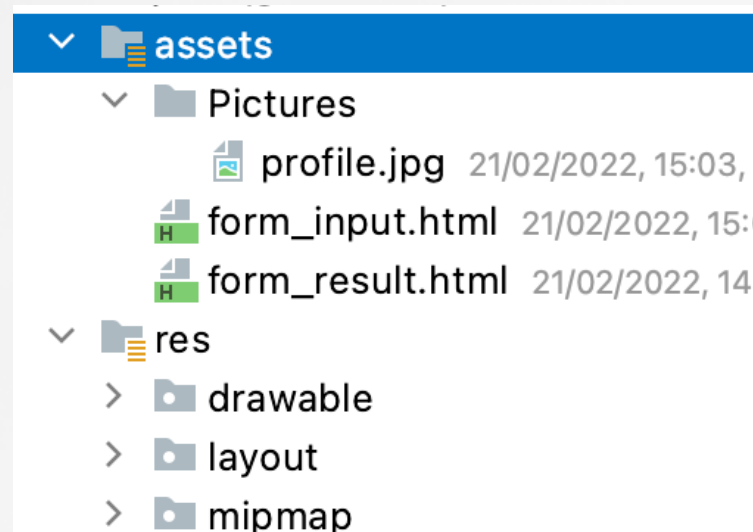
```
class MainActivity : AppCompatActivity() {  
    // ...  
    override fun onCreate(savedInstanceState: Bundle?) {  
        // ...  
        // Try demo1  
        demo1TrySpecificUrl();  
  
        handlingButtons()  
    }  
    fun handlingButtons(){  
        val backBtn : Button = findViewById(R.id.backBtn)  
        backBtn.setOnClickListener {  
            webView.goBack()  
        }  
        val forwardBtn : Button = findViewById(R.id.forwardBtn)  
        forwardBtn.setOnClickListener {  
            webView.goForward()  
        }  
        val reloadBtn : Button = findViewById(R.id.reloadBtn)  
        reloadBtn.setOnClickListener {  
            webView.reload()  
        }  
        // other buttons  
    }  
}
```

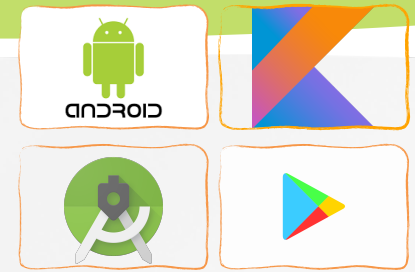


Example 2

Local pages

- » Create Assets folder and add HTML files
 - Right click > New > Folder > **Assets Folder**
 - Add ***form_input.html*** and ***form_result.html*** into that assets
 - Add picture ***profile.jpg*** into ***Pictures subs folder***





Example 2

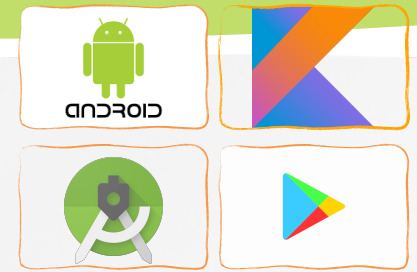
Local pages

- *form_input.html*

```
<html>
  <head> </head>
  <body>
    <FORM action="form_result.html" id=form1 method=GET name=form1>
      
      <h1>Inputting Data</h1>
      First Name:
      <br>
      <INPUT id="txtFirstName" name="txtFirstName" value="Dung"> <br>
      Last Name:
      <br>
      <INPUT id="txtLastName" name="txtLastName" value="Nguyen"> <p>
      <INPUT type="submit" value="Submit" id=submit1 name=submit1>
    </FORM>
  </body>
</html>
```

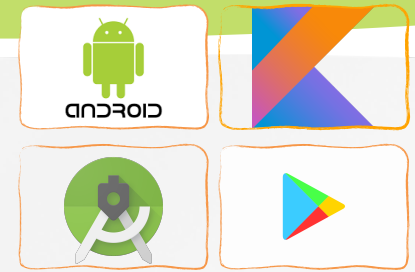
Example 2

Local pages



- *form_result.html*

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"> <html>
<head>
<title>Android & Plain HTML Demo</title>
<script language="javascript">
    function extracting(variable) {
        var query = window.location.search.substring(1);
        alert(query);
        var personName = getQueryVariable(variable)
        document.getElementById("myMsg").value = personName;
    }
    function getQueryVariable(variable) {
        var query = window.location.search.substring(1);
        var vars = query.split('&');
        for (var i=0;i<vars.length;i++) {
            var pair = vars[i].split("=");
            if ((pair[0] == variable) || (variable == 0)) {
                return pair[1];
            }
        }
        alert('Query Variable ' + variable + ' not found');
    }
</script>
</head>
<body>
    <h1>Result: Receiving Data</h1>
    <p><input TYPE="text" id="myMsg" ></p>
    <p><input TYPE="button" value="Display" onClick="extracting('txtFirstName');"></p>
    <h1>Auto receiving data from the GET query-string:</h1>
    <script>
        document.write("<p>Current time..." + new Date() );
        document.write("<p> You input the following data:");
        var querystring = window.location.search.substring(1);
        document.write("<p><b>" + querystring + "</b>");
        document.write("<p>txtLastName: " + getQueryVariable('txtLastName') );
    </script>
</body>
</html>
```

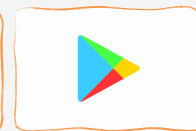


Example 2

Local pages

» *Comments*

- With Browser-Object-Model (**BOM**), class **Location** contains information about the current URL
 - The property **window.location.search** returns the query portion of a URL, including the question mark ‘?’.
 - Ex, **?txtFirstName=Dung&txtLastName=Nguyen&Submit1=Submit**
- **window.location.search.substring(0)** is **?**
txtFirstName=Dung&txtLastName=Nguyen&Submit1=Submit
- **window.location.search.substring(1)** is
txtFirstName=Dung&txtLastName=Nguyen&Submit1=Submit



Example 2

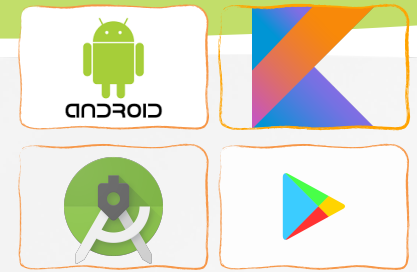
Local pages

» Main Activity

```
class MainActivity : AppCompatActivity() {  
    // . . .  
    override fun onCreate(savedInstanceState: Bundle?) {  
        // . . .  
  
        // Try demo2  
        demo2LocalPages()  
    }  
  
    fun demo2LocalPages(){  
        //1  
        webView.settings.javaScriptEnabled = true  
        //2  
        webView.webViewClient = WebViewClient()  
        //3  
        webView.loadUrl("file:///android_asset/form_input.html");  
    }  
}
```

Example 2

Local pages



» Run app

